

DocuVerse AI — Complete Interview Prep Guide

Based on your repo's README/architecture. A couple of exact numbers (chunk size, top_k, specific model names) aren't visible from outside the repo — check `backend/main1.py` and mark them below before your interview. I've flagged those spots with [CHECK CODE].

1. The 30-Second Pitch (memorize this)

"DocuVerse AI is a full-stack document intelligence platform. Users upload PDFs, and the system uses Retrieval-Augmented Generation to let them chat with the document, get AI-generated summaries, extract key insights, run semantic search, compare two documents, and export the analysis as a professional PDF report. The backend is FastAPI with LangChain orchestrating the RAG pipeline — PyPDFLoader extracts text, a recursive splitter chunks it, HuggingFace sentence-transformer embeddings vectorize the chunks, ChromaDB stores and retrieves them, and Groq's LLM generates the final grounded answer. The frontend is vanilla HTML/CSS/JS."

Practice saying this out loud in under 30 seconds without reading.

2. What Problem Does This Solve? (be ready to justify the project)

- Plain LLMs (ChatGPT, etc.) don't know about *your* private/specific documents, and they hallucinate when asked about content they weren't trained on.
 - Copy-pasting a whole document into a prompt doesn't scale — documents are often longer than the model's context window, and it's expensive/slow to feed the whole thing on every query.
 - **RAG solves this** by retrieving only the *relevant* pieces of the document at query time, and grounding the LLM's answer in that retrieved text — this reduces hallucination and lets you query documents far bigger than any context window.
-

3. System Architecture — Component by Component

```
User (Browser: HTML/CSS/JS)
  |
  v
FastAPI (backend/main1.py)
```

```

      |
      |-----|
      |         |         |
PDF Upload  AI Services  Export PDF (ReportLab)
      |
PyPDFLoader (extract text)
      |
RecursiveCharacterTextSplitter (chunk text)
      |
HuggingFace Embeddings / Sentence-Transformers (vectorize chunks)
      |
ChromaDB (store + index vectors)
      |
Retriever (similarity search on user query)
      |
Groq LLM (generate answer using retrieved context)
      |
AI-generated, context-grounded answer -> back to user

```

Why each piece is there (be ready to explain each in 1-2 sentences)

Component	Role	Why this one?
FastAPI	Web/API framework serving all endpoints	Async-native, auto-generates OpenAPI/Swagger docs (<code>/docs</code>), fast to build REST APIs in Python, type-safe via Pydantic
LangChain	Orchestration framework gluing loaders, splitters, embeddings, vector store, and LLM together	Avoids writing all the glue code (chunking, retriever interfaces, prompt templates) from scratch
PyPDFLoader	Extracts raw text from PDF pages	Standard LangChain PDF loader, page-aware (keeps page metadata)
RecursiveCharacterTextSplitter	Splits long text into overlapping chunks	Needed because embeddings work best on small, semantically coherent chunks, and because the LLM has a limited context window

Component	Role	Why this one?
HuggingFace Embeddings (Sentence-Transformers)	Converts each text chunk into a numeric vector	Open-source, runs locally/free (no API cost per embedding call, unlike OpenAI embeddings), good semantic quality for general text
ChromaDB	Vector database — stores embeddings + lets you do similarity search	Lightweight, embeddable (no separate server needed for small/medium projects), integrates natively with LangChain
Retriever	Takes the user's question, embeds it, finds nearest chunk vectors in ChromaDB	This is the "R" in RAG — turns a query into a set of relevant context chunks
Groq LLM	Generates the final natural-language answer given retrieved context + question	Groq runs open models (like Llama) on custom LPU hardware — extremely fast inference/low latency compared to typical GPU-served APIs
ReportLab	Generates downloadable PDF reports of the analysis	Python library purpose-built for programmatic PDF creation

4. The RAG Pipeline, Step by Step (this is the heart of every RAG interview)

Step 1 — Ingestion: PDF → Text

`PyPDFLoader` opens the PDF and extracts raw text per page. This is **not OCR** — it reads embedded text layers, so scanned/image-only PDFs would return empty or garbled text (this is exactly why "OCR support for scanned PDFs" is listed in your Future Enhancements — good sign you understand the current limitation).

Step 2 — Chunking: Text → Smaller Chunks

Long documents can't be embedded or fed to an LLM as one blob.

`RecursiveCharacterTextSplitter` breaks the text into chunks, trying to split on paragraph →

sentence → word boundaries in that priority order (hence "recursive") so chunks stay semantically coherent rather than being cut mid-sentence.

- **Chunk size and chunk overlap** are the two key tunable parameters. [CHECK CODE for your actual values, e.g. `chunk_size=1000, chunk_overlap=200`]
- **Why overlap matters:** if a key sentence spans the boundary between two chunks, overlap ensures it fully appears in at least one chunk instead of being split and losing meaning in both.
- **Trade-off:** smaller chunks → more precise retrieval but risk losing surrounding context; larger chunks → more context per chunk but risk diluting the vector's semantic focus (embedding "smears" over more content) and can push you over the LLM's context window when you retrieve several.

Step 3 — Embedding: Chunks → Vectors

Each chunk is passed through a HuggingFace sentence-transformer model, which outputs a fixed-length numeric vector (e.g., 384 or 768 dimensions depending on the model) that captures the chunk's semantic meaning — texts with similar meaning end up close together in vector space, even if they don't share exact words.

- [CHECK CODE for exact model name, e.g. `all-MiniLM-L6-v2`] — common choices: `all-MiniLM-L6-v2` (384-dim, fast, good default) or `all-mpnet-base-v2` (768-dim, slower, higher quality).

Step 4 — Storage: Vectors → ChromaDB

ChromaDB persists these vectors (plus the original chunk text and metadata like page number/source doc) so they can be searched later without re-embedding everything on every query.

Step 5 — Query: User Question → Retriever

When a user asks a question, the **same embedding model** embeds the question into the same vector space as the document chunks (this consistency is critical — you can't mix embedding models between ingestion and query).

Step 6 — Retrieval: Similarity Search

ChromaDB does a **nearest-neighbor search** (commonly cosine similarity, or L2/Euclidean distance depending on config) between the query vector and all stored chunk vectors, returning the **top-k** most similar chunks. [CHECK CODE for k, e.g. `k=4`]

Step 7 — Augmentation + Generation: Context + Question → Groq LLM

The retrieved chunks are stitched into a prompt template along with the user's question

(something like: "Answer the question using only the following context: {context}. Question: {question}"), and sent to the Groq-hosted LLM.

Step 8 — Response

The LLM generates an answer grounded in the retrieved context rather than purely from its own training data, which is what makes the answer specific to *your* uploaded document.

5. Feature-by-Feature Explanation (map each README feature to the pipeline)

Feature	How it likely works under the hood
PDF Upload	File saved to <code>uploads/</code> , then run through the ingestion pipeline (steps 1-4 above) to index it
PDF Viewer	Frontend renders the PDF directly in-browser (likely via an embedded viewer/iframe or PDF.js-like approach) so users can cross-reference while chatting
AI Chat	Standard RAG query flow (steps 5-8)
Summary Generation	A prompt that asks the LLM to produce a structured summary, likely using either the full extracted text or a retrieval over the whole doc plus a "summarize" instruction, possibly using a map-reduce or refine chain pattern if the doc is long (LangChain supports <code>load_summarize_chain</code> for this)
Insights Extraction	Similar to summary but the prompt is tuned to pull key facts/entities/concepts rather than a narrative summary
Semantic Search	Directly exposes the retriever step (step 6) — returns matching chunks/snippets without necessarily generating an LLM answer
Multi-Document Support	Each doc likely gets its own namespace/collection (or metadata tag) in ChromaDB so retrieval can be scoped to one document or searched across all
Document Comparison	Likely retrieves relevant chunks from <i>both</i> documents for a given topic/question and asks the LLM to synthesize similarities/differences in one prompt
Chat History	Stored server-side or client-side so previous Q&A pairs persist and can optionally be fed back as conversational memory

Feature	How it likely works under the hood
Export Reports	<code>export_service.py</code> uses ReportLab to programmatically lay out and generate a PDF containing the summary/insights/chat results

Be ready for: *"Walk me through what happens when I click 'Compare Documents.'"* → Answer using the row above, but say it conversationally, referencing the RAG pipeline explicitly.

6. Technology Deep-Dives (expect follow-ups on each)

FastAPI

- Built on Starlette (ASGI) + Pydantic — async by default, so it can handle concurrent requests (e.g., multiple users chatting) more efficiently than a sync WSGI framework like Flask.
- Auto-generates interactive Swagger docs at `/docs` from your route type hints — this is *why* your README points people to `http://127.0.0.1:8000/docs`.
- Pydantic models give you automatic request validation and clear error messages if the client sends malformed data.

LangChain

- A framework of composable building blocks: **Loaders** (get data in), **Splitters** (chunk it), **Embeddings** (vectorize), **VectorStores** (index/search), **Retrievers** (query interface over a vector store), **Chains** (compose steps, e.g., "retrieve then generate"), **Prompts** (templated instructions to the LLM).
- Common criticism to be aware of: LangChain can feel like heavy abstraction for what's conceptually simple; be ready to say you chose it for speed of development and its mature ecosystem of integrations (loaders, vector stores, LLM providers) rather than because it was strictly necessary.

ChromaDB

- An open-source, embeddable vector database — can run in-process/embedded or as a lightweight server, persists to disk (`db/` folder in your structure).
- Under the hood, approximate nearest neighbor (ANN) search (e.g., HNSW indexing) makes similarity search fast even as the number of vectors grows, avoiding a brute-force $O(n)$ comparison against every stored vector.
- **Why ChromaDB over alternatives** (be ready for "why not Pinecone/FAISS/Weaviate?"):

- *FAISS*: a library, not a full database — no built-in persistence/metadata filtering out of the box, more manual wiring.
- *Pinecone/Weaviate*: managed cloud vector DBs — great for production scale, but add cost and network latency, overkill for a portfolio/local project.
- *ChromaDB*: free, local, persists to disk, first-class LangChain integration — ideal for a project like this at this stage.

Sentence-Transformers / HuggingFace Embeddings

- These are models specifically fine-tuned (often via contrastive/siamese network training) so that semantically similar sentences produce *close* vectors — a general-purpose LLM's raw hidden states aren't optimized for this the same way.
- Cost/latency benefit: since this is a HuggingFace model (not calling OpenAI's embedding API), embeddings can run locally, meaning no per-token API cost and no external network dependency for this step.

Groq

- Groq isn't a model — it's an inference **hardware/platform** (LPU — Language Processing Unit) that runs open models (like Llama 3 or similar) at very high token-per-second speeds and low latency compared to typical GPU inference.
- Interview angle: "I chose Groq for the generation step because of its very low latency, which matters for a chat-style UX where users expect near-instant responses" — pair this with acknowledging the trade-off (dependent on Groq's hosted availability/rate limits vs. self-hosting).
- **[CHECK CODE]** — confirm exactly which model you're calling through Groq (e.g., `llama-3.1-8b-instant`, `llama-3.3-70b-versatile`, `mixtral-8x7b`) since interviewers may ask "which model, and why that size?"

ReportLab

- Lower-level Python PDF-generation library — you programmatically define text blocks, tables, styles, and layout (as opposed to converting HTML/Markdown to PDF). Good for precise control over a "professional report" look.

7. Likely Interview Questions (Basic → Advanced) with Model Answers

Basic

Q: What is RAG, and why not just fine-tune an LLM on my documents? A: RAG retrieves

relevant context at query time and feeds it into the prompt, rather than baking the knowledge into the model's weights. Fine-tuning is expensive, needs retraining every time documents change, and doesn't reliably prevent hallucination. RAG is cheaper, updates instantly when you add new documents, and grounds answers in retrievable, citable source text.

Q: What's a vector embedding? A: A fixed-length numeric representation of text (or other data) such that semantically similar inputs are close together in the vector space, typically measured by cosine similarity or Euclidean distance.

Q: Why do you need to chunk documents before embedding them? A: Because embeddings work best on focused, semantically coherent pieces of text, and because LLM context windows are limited — you can't embed or feed in an entire long document as one unit. Chunking also lets retrieval be more *precise* (surfacing exactly the relevant paragraph rather than a whole document).

Intermediate

Q: What happens if the answer to a user's question isn't in the retrieved chunks? A: Ideally the prompt instructs the LLM to say it doesn't know / isn't in the document, rather than making something up. (Good moment to be honest: if your current prompt doesn't explicitly guard against this, say "that's actually one improvement I'd make — adding an explicit instruction and a fallback for out-of-context questions.")

Q: How do you decide chunk size and overlap? A: It's a trade-off tuned empirically — too small loses context, too large dilutes the embedding's focus and risks exceeding context window budgets once you retrieve multiple chunks. **[Fill in with your actual reasoning/values once you check the code.]**

Q: How does document comparison work if each document is separately embedded? A: The retriever pulls relevant chunks from each document's collection for the same query/topic, and both sets of chunks are passed into a single prompt asking the LLM to synthesize similarities and differences.

Q: What's the difference between semantic search and your AI chat feature? A: Semantic search returns the raw matching chunks (nearest neighbors to the query) without necessarily involving the LLM in generation — it's the "R" alone. AI chat performs the full "R + G": retrieve, then have the LLM generate a natural-language answer synthesized from those chunks.

Advanced

Q: How would you evaluate whether your RAG system's answers are actually good? A: You'd want retrieval metrics (recall/precision@k — are the truly relevant chunks in the top-k retrieved?) and generation metrics (faithfulness/groundedness — does the answer

actually reflect the retrieved context, or did the LLM hallucinate beyond it?). Frameworks like RAGAS are built specifically for this.

Q: This is currently a single-user, local app. How would you scale this to many concurrent users/documents? A: Move ChromaDB from local/embedded to a proper client-server deployment (or a managed vector DB), add per-user/per-document namespacing so retrieval doesn't leak across users, add async task queues (e.g., Celery/Redis) for slow operations like PDF ingestion so uploads don't block the API, add authentication (already flagged in your Future Enhancements), and consider caching frequent queries/embeddings.

Q: What are the current limitations of this project? A: No OCR, so scanned/image PDFs won't extract text properly; no user authentication, so it's currently single-tenant; no answer citations back to the specific source page (also in your Future Enhancements as "Citation-Based Answers"); local embeddings are open-source/free but generally lower quality than something like OpenAI's `text-embedding-3-large` — a trade-off of cost vs. retrieval quality.

Q: Why HuggingFace embeddings instead of OpenAI/Azure embeddings? A: Cost and independence — no per-call API cost or external dependency for the embedding step, which matters for a project meant to run entirely locally with just one API key (Groq) required. Trade-off: proprietary embedding models are often higher-quality on benchmarks, which is explicitly why "Azure/OpenAI Embeddings" is in your own Future Enhancements list.

Q: How do you prevent hallucination in a RAG system beyond just retrieval? A: Prompt engineering (explicitly instruct the model to only use the provided context and to say "not found" if absent), lowering temperature for more deterministic/grounded output, potentially post-hoc verification (checking if generated claims are entailed by retrieved text), and citation-based answers so users can verify claims themselves.

Q: Cosine similarity vs Euclidean distance for vector search — what's the difference and when would you pick one? A: Cosine similarity measures the angle between vectors (ignores magnitude, focuses on direction/orientation — good when vector length can vary but you care about semantic direction). Euclidean (L2) distance measures straight-line distance and is sensitive to magnitude. For sentence-transformer embeddings, cosine similarity is the typical default since these models are usually optimized for it.

8. Honest Weak Points to Be Ready For (interviewers respect self-awareness)

- No OCR → can't handle scanned/image-based PDFs (you already know this — it's in your Future Enhancements).
- No authentication/multi-tenancy yet.

- No citation of exact source page/line for generated answers.
- Local embeddings trade some retrieval quality for cost/simplicity.
- Not yet cloud-deployed (also listed as future work) — be ready to discuss what deploying this would take (containerize with Docker, host ChromaDB persistently, environment variable management for the Groq key, CORS config for a separately hosted frontend).

Own these confidently — "here's what it does today, and here's exactly what I'd build next" is a strong interview answer.

9. Quick Reference — Project Structure

```
DocuVerse-AI
├── backend
│   ├── main1.py                # FastAPI app, routes, RAG pipeline wiring
│   └── services/export_service.py # ReportLab PDF report generation
├── frontend
│   ├── front_end1.html
│   ├── style.css
│   ├── app.js                 # API calls / chat logic
│   └── Ui.js                   # UI rendering/interaction logic
├── uploads/                   # stored uploaded PDFs
├── db/                         # ChromaDB persistent storage
├── requirements.txt
└── .env                       # GROQ_API_KEY
```

Run commands to have memorized:

```
python -m uvicorn backend.main1:app --reload # backend, port 8000
python -m http.server 5500                  # frontend, from /frontend
```

10. Before You Walk In — Action Items

1. Open `backend/main1.py` and confirm: chunk size, chunk overlap, top_k, embedding model name, Groq model name, exact prompt template.
2. Open `services/export_service.py` and skim how the PDF report is structured.
3. Practice the 30-second pitch (Section 1) out loud 3 times.

4. Pick your #1 "what I'd improve next" answer (Section 8) and have a specific, concrete plan for it — interviewers love a candidate who's already thought past the README's "Future Enhancements" bullet points.